TECHNICKÁ UNIVERZITA
Neurónové siete 2025/2026

Detekcia sieťových útokov pomocou neurónových sietí

Projektová správa

2025/2026

Obsah

1 Formulácia problému **3**

2 Teoretický background **4**
2.1 Multilayer Perceptron (MLP) . . . 4
2.2 Long Short-Term Memory (LSTM) . . . 4
2.3 Regularizácia a stabilizácia tréningu . . . 5

3 Popis datasetu a predspracovanie dát **5**
3.1 CIC-IDS2017 . . . 5
3.1.1 Predspracovanie dát . . . 5
3.1.2 Rozdelenie datasetu . . . 6
3.2 NSL-KDD . . . 6
3.2.1 Predspracovanie dát . . . 6

4 Metodológia **7**
4.1 MLP Architektúry . . . 7
4.1.1 Pre dataset CIC-IDS2017 . . . 7
4.1.2 Pre dataset NSL-KDD . . . 8
4.2 LSTM Architektúry . . . 9
4.2.1 Pre dataset CIC-IDS2017 . . . 9
4.2.2 Pre dataset NSL-KDD . . . 9
4.3 Infraštruktúra — HPC Perun . . . 10
4.4 Riešenie triednej nerovnováhy . . . 10
4.5 Evaluačné metriky a ich zdôvodnenie . . . 11

5 Návrh experimentov **11**
5.1 Zdôvodnenie rôznych architektúr medzi fázami . . . 12
5.2 Ablačná štúdia — zdôvodnenie . . . 13

6 Výsledky **13**
6.1 Výsledky na CIC-IDS2017 . . . 13
6.2 Vizualizácia výsledkov — CIC-IDS2017 . . . 15
6.2.1 Matice zámen . . . 15
6.2.2 Precision-Recall krivky . . . 16
6.3 Výsledky na NSL-KDD . . . 16
6.4 Súhrnné porovnanie všetkých experimentov . . . 17

7 Kritická analýza a diskusia **18**
7.1 Porovnávanie architektúr 18
7.2 Analýza výsledkov ablačnej štúdie 19
7.3 Analýza chýb 20
7.4 Technické poznatky z HPC nasadenia 20

8 Obmedzenia a možnosti ďalšej práce **21**
8.1 Obmedzenia 21
8.2 Možnosti ďalšej práce 21

9 Príspevok jednotlivých členov tímu **22**

10 Zoznam použitej literatúry **22**

1 Formulácia problému

Detekcia sieťových prienikov (Network Intrusion Detection System – NIDS) je jednou z najdôležitejších úloh v oblasti kybernetickej bezpečnosti. Cieľom NIDS je v reálnom čase identifikovať škodlivú aktivitu v počítačovej sieti – či už ide o pokus o neoprávnený prístup, distribuované odmietnutie služby (DDoS), skenovanie portov, šírenie malvéru alebo iný typ útoku.

Tradičné priemyselné riešenia sú založené na dvoch hlavných princípoch:

1. **Detekcia založená na signatúrach (signature-based):** Porovnáva sieťový prenos s databázou známych vzorov (signatúr) útokov. Tento prístup je presný na známe útoky, ale úplne zlyháva pri nových, doteraz neznámych typoch útokov (zero-day exploits).

2. **Detekcia založená na anomáliách (anomaly-based):** Vytvára model normálneho správania siete a akákoľvek odchýlka je považovaná za potenciálny útok. Hoci dokáže odhaliť nové útoky, trpí vysokým počtom falošných poplachov (false positives), čo ju robí ťažko nasaditeľnou v praxi.

Moderné prístupy sa preto čoraz viac prikláňajú k metódam strojového a hlbokého učenia, ktoré dokážu automaticky extrahovať diskriminačné vzory z dát bez potreby ručne definovaných pravidiel. Táto práca skúma, či neurónové siete – konkrétne viacvrstvový perceptrón (MLP) a dlhá krátkodobá pamäť (LSTM) – dokážu spoľahlivo rozlíšiť útočný a legitímny sieťový prenos na základe štatistických príznakov sieťových tokov.

Problém je formulovaný ako **binárna klasifikácia**:

- Trieda 0 (Benign): legitímna sieťová komunikácia (napr. bežné webové prehliadanie, e-mail, FTP).

- Trieda 1 (Attack): akýkoľvek typ sieťového útoku (DoS, DDoS, Bruteforce, Infiltration, Botnet, Port Scan, Web Attack atď.).

Hlavné výzvy, ktoré musí model riešiť, sú:

1. **Výrazná triedna nerovnováha:** V reálnych sieťach aj v použitých datasetoch (CIC-IDS2017, NSL-KDD) tvorí legitímny prenos približne 80–90% všetkých vzoriek. Model trénovaný na nevyvážených dátach má tendenciu klasifikovať takmer všetko ako "Benign", čo vedie k vysokej miere zmeškaných útokov (false negatives).

2. **Vysoká dimenzionalita:** Každý sieťový tok je opísaný 78 (CIC-IDS2017) až 41 (NSL-KDD) príznakmi – zahŕňajú základné charakteristiky (trvanie, počet paketov, počet bajtov), štatistiky dĺžok paketov, flagy TCP, počty chybových paketov a pod.

3. **Cenová asymetria chýb:** V bezpečnostných aplikáciách nie sú všetky chyby rovnako závažné. Zmeškaný útok (False Negative) je oveľa nebezpečnejší ako falošný poplach (False Positive).

4. **Časový kontext (pre LSTM):** Niektoré typy útokov (napr. pomalé skenovanie, botnetová komunikácia) nie sú identifikovateľné z jedného izolovaného sieťového toku, ale až z postupnosti viacerých tokov.

5. **Rozdielne charakteristiky datasetov:** Zatiaľ čo CIC-IDS2017 je moderný, rozsiahly dataset (2,8 milióna vzoriek), NSL-KDD je starší benchmarkový dataset (125 tisíc tréningových vzoriek) s inou distribúciou príznakov a typov útokov.

Cieľom tejto práce je navrhnúť, implementovať a experimentálne zhodnotiť rôzne architektúry neurónových sietí (MLP, LSTM) na dvoch odlišných datasetoch, pričom sa zameriavame nielen na celkovú presnosť, ale najmä na minimalizáciu zmeškaných útokov a schopnosť generalizácie na neznáme typy útokov. Súčasťou je aj ablačná štúdia, ktorá skúma vplyv kapacity modelu a dĺžky časového okna na výsledný výkon.

2 Teoretický background

2.1 Multilayer Perceptron (MLP)

MLP je základná architektúra doprednej neurónovej siete skladajúca sa z plne prepojených vrstiev. Pre každý neurón v skrytej vrstve platí: $y = f(Wx + b)$, kde W sú váhy, b je bias a f je aktivačná funkcia. V tejto práci používame ReLU (Rectified Linear Unit) pre skryté vrstvy a Sigmoid pre výstupnú vrstvu binárnej klasifikácie.

MLP slúži ako baseline model — jeho jednoduché usporiadanie umožňuje rýchly tréning a interpretáciu, no nezachytáva časové vzťahy medzi paketmi. Pri spracovaní sekvencií (napr. sieťovej komunikácie) by MLP musel dostať na vstup celé časové okno naraz, čo neumožňuje dynamické prispôsobovanie sa postupne prichádzajúcim dátam a ignoruje závislosti medzi paketmi vzdialenými v čase.

2.2 Long Short-Term Memory (LSTM)

Motivácia: Problém zanikajúceho gradientu

Rekurentné neurónové siete (RNN) majú teoreticky schopnosť uchovávať informácie o minulých udalostiach v podobe aktivácií (tzv. krátkodobá pamäť). V praxi však trénovanie RNN metódami spätného šírenia chyby v čase (Backpropagation Through Time, BPTT) naráža na zásadný problém: pri spätnom šírení chybových signálov do minulosti dochádza k ich exponenciálnemu zosilneniu alebo, častejšie, k exponenciálnemu zániku. Hochreiter (1991) vo svojej diplomovej práci analyticky ukázal, že veľkosť gradientu pri spätnom prechode cez q časových krokov je ohraničená výrazom $n(f'_{\max}|W|_A)^q$, kde $f'_{\max}$ je maximálna derivácia aktivačnej funkcie (napr. 0.25 pre sigmoid) a $|W|_A$ je norma matice váh. Pre bežné hodnoty váh (absolútna hodnota < 4) tak gradient s rastúcim q rýchlo klesá k nule. To znemožňuje sieti naučiť sa závislosti, ktoré vyžadujú uchovanie informácie cez viac než 10 časových krokov.

Architektúra LSTM a konštantný tok chyby

Hochreiter a Schmidhuber (1997) navrhli architektúru nazvanú Long Short-Term Memory (LSTM), ktorá tento problém rieši zavedením špecializovaných pamäťových buniek (memory cells). Každá bunka obsahuje *constant error carousel* (CEC) – lineárnu jednotku so samospätnou väzbou s váhou 1.0. Vďaka tomu môže chybový signál prúdiť cez vnútorný stav bunky $s_{c_j}(t)$ s konštantnou veľkosťou, bez zániku či explózie.

Aby sa však zabránilo rušeniu uloženej informácie nežiaducimi vstupmi a výstupmi, LSTM zavádza dve multiplikatívne brány (gate units):

- **Vstupná brána** in_j – riadi, kedy sa nová informácia zapíše do pamäťovej bunky. Tým sa rieši *input weight conflict* – rovnaké vstupné váhy by inak museli súčasne slúžiť na zapamätanie aj ignorovanie rôznych vstupov.
- **Výstupná brána** out_j – riadi, kedy sa obsah bunky sprístupní zvyšku siete. Výstupná brána rieši *output weight conflict* – chráni ostatné jednotky pred rušivými vplyvmi pamäťovej bunky v čase, keď jej obsah nie je potrebný.

Formálne, pre vnútorný stav bunky c_j^v platí:

$$s_{c_j^v}(t) = s_{c_j^v}(t-1) + y^{in_j}(t) \cdot g(net_{c_j^v}(t)), \qquad y^{c_j^v}(t) = y^{out_j}(t) \cdot h\big(s_{c_j^v}(t)\big).$$

Tréning LSTM využíva modifikovanú verziu BPTT s *truncated backpropagation* – chybové signály sa nešíria ďalej do minulosti, keď opustia brány alebo vstup bunky. To zaisťuje, že vnútri CEC zostáva tok chyby konštantný, a zároveň je výpočtová zložitosť na časový krok $O(W)$, kde W je počet váh – rovnako ako pri BPTT, ale na rozdiel od RTRL ($O(W^2)$).

LSTM v kontexte detekcie sieťových útokov

Sieťová komunikácia je prirodzene sekvenčná: útočník vykonáva skenovanie, odosiela škodlivé payloady, následne nadväzuje spojenie – tieto udalosti sú rozložené v čase. LSTM vďaka svojej schopnosti uchovávať informáciu po stovky časových krokov dokáže modelovať správanie útočníka aj v prípade, že medzi podozrivým paketom a jeho následkom uplynie dlhý čas. Navyše, LSTM je robustná voči šumu a distribuovaným reprezentáciám – čo je kľúčové pre reálne dáta ako CIC-IDS2017, ktoré obsahujú veľa náhodných alebo irelevantných paketov (distractor symbols) zmiešaných s útočnými vzormi.

2.3 Regularizácia a stabilizácia tréningu

Dropout náhodne deaktivuje určitý podiel neurónov počas tréningu, čím zabraňuje overfittingu. Batch Normalization normalizuje aktivácie každej vrstvy, čo stabilizuje tréning a umožňuje vyššie learning rates. Oba mechanizmy sú obzvlášť dôležité pri práci s CIC-IDS2017, ktorý obsahuje veľké množstvo vzoriek s výraznými hodnotami príznakov.

Pri LSTM sa často používa aj *gradient clipping* (orezanie gradientov) na potlačenie prípadných explózií, a *inicializácia biasov výstupných brán na záporné hodnoty* (napr. -2, -4). Záporná počiatočná hodnota výstupnej brány potláča aktivitu pamäťových buniek na začiatku tréningu, čím sa predchádza *abuse problem* – sieť sa nespolieha na pamäťové bunky ako na obyčajné bias jednotky.

3 Popis datasetu a predspracovanie dát

3.1 CIC-IDS2017

Dataset CIC-IDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) bol vytvorený Univerzitou v New Brunswicku. Obsahuje zachytené sieťové toky z reálnych scenárov útokov realizovaných v kontrolovanom prostredí počas 5 pracovných dní.

Vlastnosť	Popis
Počet záznamov	∼2,8 milióna sieťových tokov
Po čistení (NaN/Inf)	2 827 876 vzoriek (odstránených 2 867)
Počet príznakov	78 → 69 po odstránení konštantných stĺpcov
Triedy	Benign: 2 271 320 (80.3%) | Attack: 556 556 (19.7%)
Typy útokov	DoS, DDoS, Brute Force, XSS, SQL Injection, Infiltration, Botnet, Port Scan
Formát	8 CSV súborov (Monday–Friday, ∼170k–693k riadkov/súbor)

Tabuľka 1: Charakteristika datasetu CIC-IDS2017

3.1.1 Predspracovanie dát

Pipeline predspracovania pozostáva z nasledujúcich krokov:

- Načítanie a zlúčenie: všetky CSV súbory z priečinka MachineLearningCVE sú načítané a konkatenované do jedného DataFrame.
- Čistenie názvov stĺpcov: odstránenie nadbytočných medzier zo záhlaví (typický problém CIC-IDS2017).
- Náhrada nekonečných hodnôt: hodnoty +inf a -inf sú nahradené NaN a následne riadky s NaN sú odstránené. Príznaky ako `Flow Bytes/s` môžu obsahovať inf pri nulových časoch.

- Odstránenie duplikátnych stĺpcov: stĺpec `Fwd Header Length.1` je duplikátom a je vymazaný.
- Binárizácia náveští: BENIGN → 0, akýkoľvek útok → 1.
- Odstránenie konštantných príznakov: príznaky s nulovou variabilitou nenesú informáciu (odstránených 8 stĺpcov).
- Normalizácia MinMaxScaler: škálovanie všetkých príznakov do rozsahu [0, 1].

3.1.2 Rozdelenie datasetu

Množina	Podiel	Veľkosť (MLP)	Účel
Tréningová	64%	1 809 840	Učenie váh modelu
Validačná	16%	452 460	Ladenie hyperparametrov
Testovacia	20%	565 576	Finálna nezaujatá evaluácia

Tabuľka 2: Rozdelenie datasetu CIC-IDS2017

Pre MLP modely bolo použité stratifikované náhodné rozdelenie (`shuffle=True`). Pre LSTM modely bol implementovaný *per-day stratifikovaný split* — z každého dňa (CSV súboru) sa nezávisle vyčlenilo 64 %/16 %/20 % riadkov v chronologickom poradí. Tým každá množina obsahuje záznamy zo všetkých dní a teda všetkých typov útokov, pričom temporálny poriadok vnútri každého dňa je zachovaný.

3.2 NSL-KDD

Dataset NSL-KDD je štandardným benchmarkom v oblasti detekcie sieťových prienikov. Ide o vylepšenú verziu pôvodného datasetu KDD'99, z ktorého boli odstránené redundantné záznamy, čím sa zabránilo skresleniu pri učení modelov.

Vlastnosť	Popis
Počet tréningových záznamov	125 973 (súbor `KDDTrain+.csv`)
Počet testovacích záznamov	22 544 (súbor `KDDTest+.csv`)
Počet príznakov	41 (38 numerických, 3 kategorické)
Typy útokov	DoS, Probe, R2L (Root to Local), U2R (User to Root)
Triedna nerovnováha (Test)	∼43% Benign (9 710), ∼57% Attack (12 833)
Formát	Predpripravené oddelené CSV súbory

Tabuľka 3: Charakteristika datasetu NSL-KDD

3.2.1 Predspracovanie dát

Pipeline predspracovania pre NSL-KDD sa mierne líši z dôvodu prítomnosti kategorických premenných:

- Načítanie stĺpcov: Názvy stĺpcov boli extrahované zo súboru `Field Names.csv` a priradené k dátam, keďže pôvodné CSV súbory neobsahujú hlavičku.
- Odstránenie nepotrebných atribútov: Stĺpec `difficulty` bol vymazaný, keďže neslúži ako predikčný príznak.
- Binárizácia náveští: Záznamy s náveštím `normal` → 0, akýkoľvek iný typ útoku → 1.

- Kódovanie kategorických príznakov: Textové atribúty (`protocol_type`, `service`, `flag`) boli transformované na číselné hodnoty pomocou `LabelEncoder`. Kódovanie prebehlo na spojených dátach (train + test), aby sa predišlo chybám z neznámych kategórií v testovacej množine.
- Normalizácia: Použitie `MinMaxScaler` na preškálovanie všetkých numerických hodnôt do intervalu [0, 1].
- Generovanie časových okien (pre LSTM): dáta transformované do 3D tenzorov s veľkosťou okna (*window size*) = 5 záznamov (`create_windows` funkcia).

Špecifikom testovacej množiny NSL-KDD je, že obsahuje špecifické typy útokov, ktoré sa v tréningovej množine nenachádzajú. Tento fakt robí evaluáciu na tomto datasete obzvlášť prísnou pre testovanie generalizačných schopností neurónových sietí. Bol implementovaný *per-class ordered split*: z každej triedy (Normal/Attack) nezávisle vyčlenených 80 % tréningových a 20 % validačných vzoriek v poradí záznamu v CSV súbore. Výsledok: 100 778 tréningových vzoriek, 25 195 validačných vzoriek.

4 Metodológia

4.1 MLP Architektúry

4.1.1 Pre dataset CIC-IDS2017

Architektúra MLP Baseline je navrhnutá ako kompaktná, ale dostatočne expresívna sieť pre vysokodimenzionálne tabuľkové dáta. Skladá sa z nasledujúcich vrstiev:

$$\text{Input}(69) \to \text{Dense}(64, \text{ReLU}) \to \text{BatchNorm} \to \text{Dropout}(0.2) \to \text{Dense}(32, \text{ReLU}) \to \text{Dense}(1, \text{Sigmoid}) \tag{1}$$

Vstupná vrstva. Po predspracovaní má každý sieťový tok dimenziu $d = 69$ príznakov. Vstupná vrstva teda prijíma vektor $\mathbf{x} \in [0, 1]^{69}$ pre každú vzorku nezávisle — MLP neuvažuje žiadny sekvenčný kontext medzi po sebe idúcimi tokmi.

Prvá skrytá vrstva — Dense(64, ReLU). Plne prepojená vrstva s 64 neurónmi a aktivačnou funkciou ReLU:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{64\times 69} \tag{2}$$

ReLU ($f(z) = \max(0, z)$) bola zvolená pre svoju výpočtovú jednoduchosť a efektívne potlačenie problému zanikajúceho gradientu v porovnaní so sigmoid/tanh aktiváciami v skrytých vrstvách.

Batch Normalization. Po prvej skrytej vrstve je zaradená vrstva Batch Normalization, ktorá normalizuje aktivácie po každom mini-batchi:

$$\hat{h}_{1,i} = \frac{h_{1,i} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \cdot \gamma + \beta \tag{3}$$

kde $\mu_{\mathcal{B}}$ a $\sigma_{\mathcal{B}}^2$ sú priemer a rozptyl mini-batchu, γ a β sú trénovateľné parametre škálovania a posunutia, $\varepsilon = 10^{-3}$ zabraňuje deleniu nulou. Tento mechanizmus je obzvlášť dôležitý pre CIC-IDS2017, kde niektoré príznaky (napr. `Flow Bytes/s`, `Packet Length Std`) nadobúdajú extrémne hodnoty aj po normalizácii, čo by bez BatchNorm destabilizovalo tréning.

Dropout(0.2). Regularizačná vrstva s pravdepodobnosťou deaktivácie $p = 0.2$ náhodne vypína 20 % neurónov počas každého tréningového kroku. Tým sa predchádza ko-adaptácii neurónov a overfittingu na tréningovú množinu. Počas inferencie sú všetky neuróny aktívne a ich výstupy sú škálované faktorom $(1 - p)$.

Druhá skrytá vrstva — Dense(32, ReLU). Druhá vrstva redukuje reprezentáciu na 32 neurónov:

$$\mathbf{h}_2 = \mathrm{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{32\times 64} \tag{4}$$

Postupné zužovanie dimenzie ($69 \to 64 \to 32$) núti sieť extrahovať čoraz abstraktnejšie reprezentácie príznakov sieťového toku.

Výstupná vrstva — Dense(1, Sigmoid).

$$\hat{y} = \sigma(\mathbf{w}_{\text{out}}^\top \mathbf{h}_2 + b_{\text{out}}) = \frac{1}{1 + e^{-(\mathbf{w}_{\text{out}}^\top \mathbf{h}_2 + b_{\text{out}})}} \in (0, 1) \tag{5}$$

Binárna predikcia sa určí prahovaním: $\hat{c} = \mathbf{1}[\hat{y} > 0.5]$.

Optimalizátor a stratová funkcia. Model je trénovaný optimalizátorom Adam s predvolenou rýchlosťou učenia $\eta = 10^{-3}$ a binárnou krížovou entropiou ako stratovou funkciou:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} w_{c_i}\left[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\right] \tag{6}$$

kde w_{c_i} je váha triedy vzorky i (Benign: $w_0 = 0.621$, Attack: $w_1 = 2.571$).

Celkový počet parametrov.

$$(69{\cdot}64{+}64){+}(64{\cdot}4{\cdot}2){+}(64{\cdot}32{+}32){+}(32{+}1) = 4{,}545{+}512{+}2{,}080{+}33 = 7{,}170 \text{ parametrov} \tag{7}$$

4.1.2 Pre dataset NSL-KDD

Pre menší benchmarkový dataset NSL-KDD používame zjednodušenú verziu MLP bez Batch Normalization. Táto voľba reflektuje nižšiu komplexitu a menší rozsah datasetu (100 778 tréningových vzoriek oproti 1 809 840 pri CIC-IDS2017). Architektúra má tvar:

$$\text{Input}(41) \to \text{Dense}(64, \text{ReLU}) \to \text{Dense}(32, \text{ReLU}) \to \text{Dense}(1, \text{Sigmoid}) \tag{8}$$

Celkový počet parametrov.

$$(41 \cdot 64 + 64) + (64 \cdot 32 + 32) + (32 \cdot 1 + 1) = 2{,}688 + 2{,}080 + 33 = 4{,}801 \text{ parametrov} \tag{9}$$

Model je teda takmer o polovicu menší ako jeho CIC-IDS2017 ekvivalent (4 801 vs. 7 170 parametrov), čo zodpovedá nižšej vstupnej dimenzii a absencii BatchNorm parametrov.

Špecifikum testovacej množiny NSL-KDD. Kľúčovou vlastnosťou tohto benchmarku je, že testovacia množina (`KDDTest+.csv`) obsahuje typy útokov, ktoré sa v tréningovej množine vôbec nevyskytujú. Tento fakt priamo vysvetľuje nižší Recall dosiahnutý na testovacej množine.

4.2 LSTM Architektúry

4.2.1 Pre dataset CIC-IDS2017

Na rozdiel od MLP, ktorý klasifikuje každý sieťový tok ako izolovanú vzorku, LSTM model spracúva *sekvencie* po sebe idúcich tokov a dokáže zachytiť temporálne vzory útočného správania rozložené v čase. Architektúra má tvar:

$$\begin{aligned}\text{Input}(10,\,69) &\to \text{LSTM}(64,\ \text{return_seq=True}) \to \text{BatchNorm} \to \text{Dropout}(0.2)\\ &\to \text{LSTM}(32) \to \text{Dense}(16,\ \text{ReLU}) \to \text{Dense}(1,\ \text{Sigmoid})\end{aligned} \tag{10}$$

Vstup — časové okná. Namiesto jednotlivých vektorov $\mathbf{x} \in \mathbb{R}^{69}$ prijíma LSTM trojrozmerný tenzor $\mathbf{X} \in \mathbb{R}^{B\times T\times F}$, kde B je veľkosť mini-batchu, $T = 10$ je dĺžka časového okna a $F = 69$ je počet príznakov. Každá vzorka v batchi teda predstavuje postupnosť 10 po sebe idúcich sieťových tokov, pričom mítka $y \in \{0, 1\}$ patrí poslednému toku v okne (index $T - 1$).

Časové okná sú generované funkciou `create_sequences`: pre každý index $i \in \{0, \ldots, N - T\}$ sa vytvorí dvojica $(\mathbf{X}_{i:i+T},\ y_{i+T-1})$.

Prvá LSTM vrstva — LSTM(64, return_sequences=True). LSTM bunka v čase t prijíma vstup $\mathbf{x}_t \in \mathbb{R}^{69}$ a predchádzajúci skrytý stav $\mathbf{h}_{t-1} \in \mathbb{R}^{64}$, pričom aktualizuje svoj vnútorný stav podľa rovníc:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \qquad \text{(forget gate)} \tag{11}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \qquad \text{(input gate)} \tag{12}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \qquad \text{(candidate cell)} \tag{13}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \qquad \text{(cell state update)} \tag{14}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \qquad \text{(output gate)} \tag{15}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \qquad \text{(hidden state)} \tag{16}$$

kde $\odot$ označuje Hadamardov (prvkový) súčin a σ je sigmoická funkcia.

Počet parametrov prvej LSTM vrstvy:

$$4 \times \big[(69 + 64) \times 64 + 64\big] = 4 \times 8\,576 = 34\,304 \tag{17}$$

Druhá LSTM vrstva — LSTM(32, return_sequences=False). Druhá vrstva s 32 bunkami vracia iba posledný skrytý stav $\mathbf{h}_{10} \in \mathbb{R}^{32}$, ktorý reprezentuje komprimovanú temporálnu sumarizáciu celej 10-krokovej sekvencie.

Počet parametrov druhej LSTM vrstvy:

$$4 \times \big[(64 + 32) \times 32 + 32\big] = 4 \times 3\,104 = 12\,416 \tag{18}$$

Celkový počet parametrov.

$$\underbrace{34\,304}_{\text{LSTM}_1} + \underbrace{512}_{\text{BatchNorm}} + \underbrace{12\,416}_{\text{LSTM}_2} + \underbrace{528}_{\text{Dense(16)}} + \underbrace{17}_{\text{Dense(1)}} = 49\,777 \text{ parametrov} \tag{19}$$

4.2.2 Pre dataset NSL-KDD

Pre benchmarkový dataset NSL-KDD bola zvolená zjednodušená LSTM architektúra s jednou rekurentnou vrstvou a kratším časovým oknom:

$$\text{Input}(5,\,41) \to \text{LSTM}(64,\ \text{return_seq=False}) \to \text{Dropout}(0.2) \to \text{Dense}(1,\ \text{Sigmoid}) \tag{20}$$

Rozdiel oproti CIC-IDS2017 variante je zámerný a vychádza z troch faktorov:

- **Veľkosť datasetu:** NSL-KDD obsahuje 100 778 tréningových vzoriek oproti 1 809 840 pri CIC-IDS2017 — teda iba ∼5,6 % objemu.
- **Dimenzia príznakov:** 41 príznakov (oproti 69) predstavuje jednoduchší vstupný priestor.
- **Povaha datasetu:** NSL-KDD neobsahuje explicitné časové razítka ani garantovaný chronologický poriadok záznamov — sekvencie window=5 sú skôr lokálnym kontextom než skutočnou temporálnou históriou.

Počet parametrov LSTM vrstvy: $4 \times \big[(41 + 64) \times 64 + 64\big] = 27\,136$. Celkovo: $27\,136 + 65 = 27\,201$ parametrov.

Model	Parametre	Pomer vs. MLP Baseline
MLP Baseline (CIC)	7 170	1.0×
MLP Benchmark (NSL)	4 801	0.67×
LSTM Benchmark (NSL)	27 201	3.8×
LSTM Advanced (CIC)	49 777	6.9×

Tabuľka 4: Porovnanie počtu parametrov všetkých architektúr

4.3 Infraštruktúra — HPC Perun

Experimenty boli spustené na superpočítači HPC Perun (SLURM cluster) s nasledujúcou konfiguráciou:

- GPU: NVIDIA H200 (140 831 MB VRAM, Compute Capability 9.0)
- CUDA: verzia 12.9, cuDNN: 9.2.1.0
- TensorFlow: 2.20.0, Keras: 3.10.0
- Prostredie: Python 3.9, pip lokálna inštalácia (`tensorflow[and-cuda]`)
- Tréningový čas MLP: ∼5 s/epocha | LSTM: ∼24 s/epocha
- Celkový čas behu: ∼15 minút (18:53–19:08, 1. mája 2026)

4.4 Riešenie triednej nerovnováhy

Dataset	Benign/Normal	Attack	Pomer	Imbalance ratio
CIC-IDS2017 (train)	1 457 644	352 196	80.3 % / 19.7 %	4.1 : 1
NSL-KDD (train)	67 343	58 630	53.5 % / 46.5 %	1.1 : 1
NSL-KDD (test)	9 710	12 833	43.1 % / 56.9 %	1 : 1.3

Tabuľka 5: Distribúcia tried v použitých datasetoch

Distribúcia tried v datasetoch.

Matematická formulácia váh tried.

$$w_c = \frac{N}{K \cdot N_c}, \quad c \in \{0, 1\} \tag{21}$$

Pre CIC-IDS2017:

$$w_0\,(\text{Benign}) = \frac{1\,809\,840}{2 \times 1\,457\,644} = 0.621 \tag{22}$$

$$w_1\,(\text{Attack}) = \frac{1\,809\,840}{2 \times 352\,196} = 2.571 \tag{23}$$

Pre NSL-KDD: $w_0 = 0.935$, $w_1 = 1.074$ — takmer symetrické okolo 1.0, čo potvrdzuje takmer vyváženú distribúciu tréningových dát tohto benchmarku.

4.5 Evaluačné metriky a ich zdôvodnenie

		Predikovaná trieda	
		Benign (0)	**Attack (1)**
Skutočná trieda	**Benign (0)**	TN	FP
	Attack (1)	FN	TP

Tabuľka 6: Matica zámen pre binárnu klasifikáciu útokov

Základné pojmy — matica zámen.

Prehľad metrík.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{24}$$

$$\text{F1} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}, \qquad \text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} = 1 - \text{Recall} \tag{25}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \qquad \text{AUC} = P(\hat{y}_{\text{attack}} > \hat{y}_{\text{benign}}) \tag{26}$$

FNR (False Negative Rate) je primárnou bezpečnostnou metrikou — každý prípad FN predstavuje útok, ktorý prešiel detekčným systémom bez povšimnutia. **FPR** (False Positive Rate) degraduje použiteľnosť systému v praxi — administrátori sú zahltení falošnými poplachmi (*alert fatigue*). **ROC-AUC** je nezávislá od prahu rozhodnutia a od distribúcie tried. Celková presnosť (Accuracy) je pri nerovnomerne rozdelených triedach zavádzajúca — model predikujúci výhradne triedu Benign by dosiahol Accuracy = 80.3 % na CIC-IDS2017 pri nulovej schopnosti detekcie útokov.

5 Návrh experimentov

Experimenty sú rozdelené do dvoch fáz podľa použitého datasetu. Fáza 1 využíva CIC-IDS2017 pre analýzu v prostredí veľkých dát na GPU infraštruktúre HPC Perun, Fáza 2 využíva NSL-KDD pre benchmarkové overenie generalizačných schopností architektúr.

Experiment	Model	Konfigurácia	Cieľ
Fáza 1: CIC-IDS2017 — 1 809 840 tréningových vzoriek, 69 príznakov, GPU NVIDIA H200			
E1: MLP Baseline	MLP	dropout=0.2, units=(64,32), BatchNorm, epochs=15, batch=256	Referenčná línia bez sekvenčného kontextu.
E2: MLP Large (ablation)	MLP	dropout=0.4, units=(128,64), BatchNorm, epochs=15, batch=256	Vplyv kapacity — H1 vs H2.
E3: LSTM Advanced	LSTM	window=10, lstm=(64,32), BatchNorm, Dropout=0.2, epochs=15, batch=256, per-day split	Sekvenčný kontext 10 tokov.
E4: LSTM Window=5 (ablation)	LSTM	window=5, lstm=(64,32), BatchNorm, Dropout=0.2, epochs=15, batch=256, per-day split	Vplyv dĺžky okna — úspora pamäte.
Fáza 2: NSL-KDD — 100 778 tréningových vzoriek (80% z KDDTrain+), 41 príznakov, CPU, per-class order			
E5: MLP Baseline	MLP	units=(64,32), bez BatchNorm, epochs=10, batch=128	Referencia na benchmarkovom datasete.
E5b: MLP Large (ablation)	MLP	units=(128,64), Dropout=0.3, epochs=10, batch=128	Vplyv kapacity MLP na NSL-KDD — analógia E2 pre Fázu 1.
E6: LSTM Baseline	LSTM	window=5, lstm=(64), Dropout=0.2, epochs=10, batch=128	Sekvenčná analýza na i.i.d. datasete.
E6b: LSTM Large (ablation)	LSTM	window=5, lstm=(128), Dense=32, Dropout=0.2, epochs=10, batch=128	Vplyv kapacity LSTM na i.i.d. datasete.

Tabuľka 7: Komplexný návrh experimentov (8 experimentov celkovo)

5.1 Zdôvodnenie rôznych architektúr medzi fázami

Dataset + Model	Tréning. vzorky	Parametre	Pomer	Riziko
CIC-IDS2017 + MLP Baseline	1 809 840	7 170	252 000 : 1	Nízke
CIC-IDS2017 + LSTM Advanced	1 809 840	49 777	36 000 : 1	Nízke
NSL-KDD + MLP Benchmark	100 778	4 801	21 000 : 1	Stredné
NSL-KDD + LSTM Benchmark	100 773	27 201	3 700 : 1	Vyššie

Tabuľka 8: Pomer tréningových vzoriek k počtu parametrov modelu

Objem dát a riziko overfittingu.

Rozdelenie datasetu — shuffle vs. per-day stratifikovaný split. Pre **MLP modely** na oboch datasetoch bolo použité náhodné stratifikované premiešanie (`shuffle=True, stratify=y`).

Pre **LSTM modely na CIC-IDS2017** boli testované tri prístupy:

1. **Shuffle=True:** náhodné premiešanie eliminuje temporálne závislosti. ROC-AUC ≈ 0.5, TP $= 0$. *Nepoužiteľné.*
2. **Globálny chronologický split:** domain shift — val_loss rástol z 0.093 na 1.635. *Nepoužiteľné.*
3. **Per-day stratifikovaný split (zvolená metóda):** z každého dňa nezávisle 64 %/16 %/20 % chronologicky. ROC-AUC = 0.9821.

Metóda splitu	ROC-AUC	F1	FNR
Shuffle=True	≈ 0.500	≈ 0	$\approx 100\,\%$
Globálny chrono. split	0.9029	0.681	$\sim 21\,\%$
Per-day split (finálny)	**0.9821**	**0.9477**	$\sim$ **9** %
MLP Baseline (shuffle=True)	0.9985	0.9504	0.55 %

Tabuľka 9: Vplyv metodológie rozdelenia datasetu na výkon LSTM Advanced (E3)

Rozdiel medzi globálnym a per-day splitom (ΔROC-AUC $= +0.079$) je väčší ako rozdiel medzi MLP a LSTM pri rovnakom splite (ΔROC-AUC $= 0.016$). *Metodológia rozdelenia dát má väčší vplyv na výsledky ako voľba architektúry.*

5.2 Ablačná štúdia — zdôvodnenie

Vplyv kapacity modelu (E1 vs. E2 a E5 vs. E5b). Ablačné experimenty menia výhradne kapacitu siete a Dropout pri zachovaní všetkých ostatných podmienok. Hypotézy:

- **H1 (underfitting):** väčšia sieť výrazne prekoná Baseline.
- **H2 (saturácia):** výsledky porovnateľné alebo horšie.

Vplyv temporálneho kontextu (E1/E2 vs. E3 a E5/E5b vs. E6/E6b). Porovnaním LSTM s MLP modelmi izolujeme čistý prínos sekvenčných informácií.

Vplyv veľkosti okna (E3 vs. E4). Skrátenie okna z 10 na 5 záznamov testuje kompromis medzi temporálnym kontextom a pamäťovou náročnosťou. Kratšie okno šetrí ≈ 2.5 GB RAM.

6 Výsledky

6.1 Výsledky na CIC-IDS2017

Experimenty boli spustené na GPU NVIDIA H200 (HPC Perun, 1. mája 2026, 18:53–19:08). Testovacia množina obsahuje 565 576 vzoriek (523 421 Benign, 42 154 Attack).

Model	Accuracy	F1 (Attack)	ROC-AUC	FNR	Avg. Prec.
E1: MLP Baseline	0.98	0.9504	0.9985	0.55 %	0.9941
E2: MLP Large	0.98	0.9490	0.9986	0.60 %	0.9942
E3: LSTM Win=10	0.99	0.9477	0.9821	9.24 %	0.9482
E4: LSTM Win=5	0.99	0.9455	0.9825	9.10 %	0.9450

Tabuľka 10: Výsledky na testovacej množine CIC-IDS2017 — per-day stratifikovaný split, 1. mája 2026

Model	TP	TN	FP	FN	FNR
E1: MLP Baseline	110 700	443 329	10 936	611	0.55 %
E2: MLP Large	110 638	443 057	11 208	673	0.60 %
E3: LSTM Win=10	38 261	523 086	330	3 893	9.24 %
E4: LSTM Win=5	38 316	522 840	581	3 838	9.10 %

Tabuľka 11: Matica zámen na testovacej množine CIC-IDS2017 — hodnoty z confusion matrix grafov

Detailná analýza chýb — CIC-IDS2017. Kľúčovým pozorovaním je diametrálne odlišné rozloženie chýb medzi MLP a LSTM: MLP Baseline zmeškal 611 útokov (FNR = 0.55 %), ale generoval 10 936 falošných poplachov (FPR = 2.41 %). LSTM Win=5 zmeškal 3 838 útokov (FNR = 9.10 %), ale generoval iba 581 falošných poplachov (FPR = 0.11 %) — teda 23× menej ako MLP Baseline.

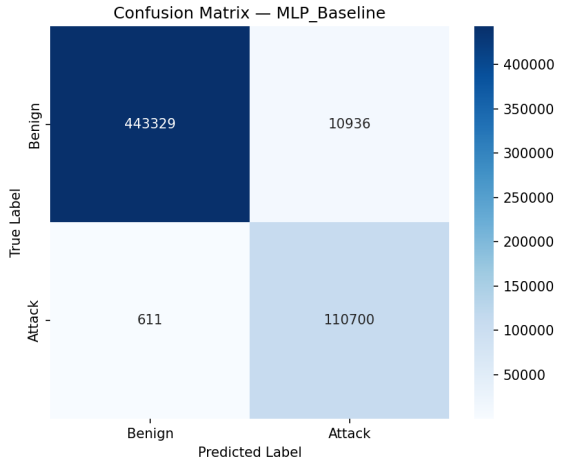
Model	Epocha 1 (acc)	Epocha 15 (acc)	Val acc (best)	Čas/epocha
E1: MLP Baseline	0.937	0.979	0.981	∼5 s
E2: MLP Large	0.934	0.979	0.983	∼5 s
E3: LSTM Win=10	0.914	0.999	0.999	∼24 s
E4: LSTM Win=5	0.914	0.999	0.999	∼24 s

Tabuľka 12: Priebeh tréningu modelov na CIC-IDS2017

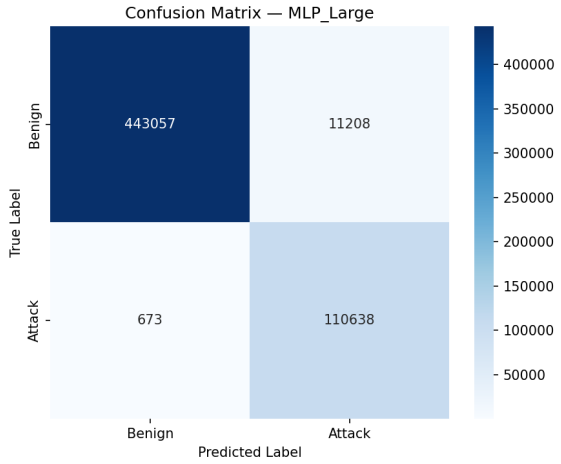
Priebeh tréningu.

6.2 Vizualizácia výsledkov — CIC-IDS2017

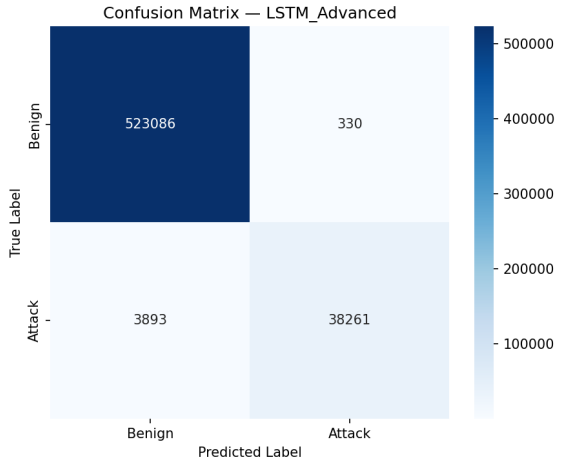
6.2.1 Matice zámen



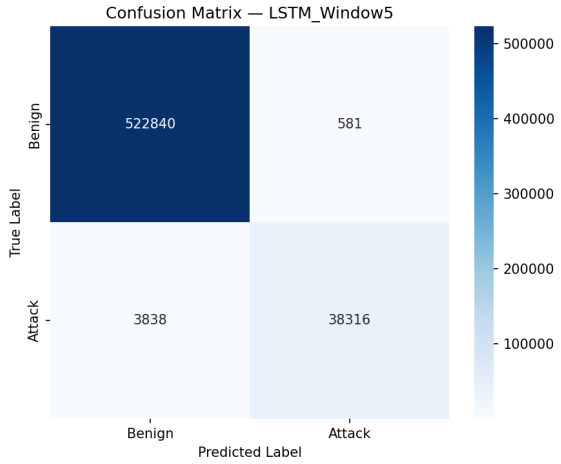
Obr. 1: E1: MLP Baseline — TN=443 329 | FP=10 936 | FN=611 | TP=110 700



Obr. 2: E2: MLP Large — TN=443 057 | FP=11 208 | FN=673 | TP=110 638

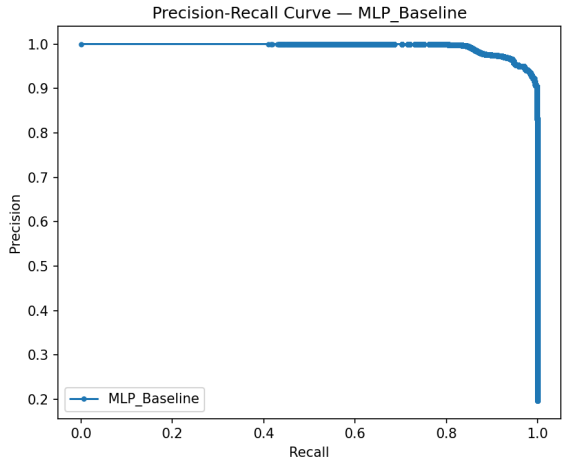


Obr. 3: E3: LSTM Win=10 — TN=523 086 | FP=330 | FN=3 893 | TP=38 261

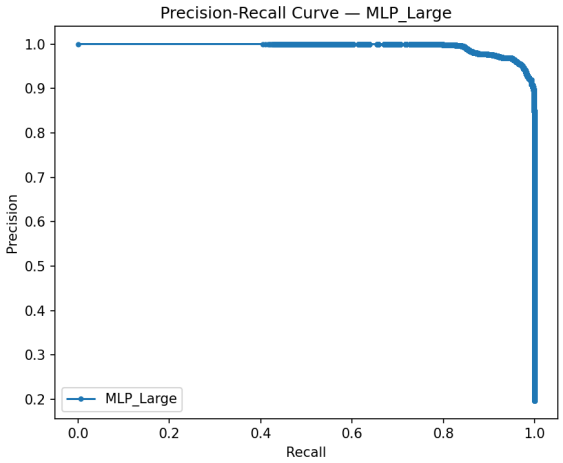


Obr. 4: E4: LSTM Win=5 — TN=522 840 | FP=581 | FN=3 838 | TP=38 316

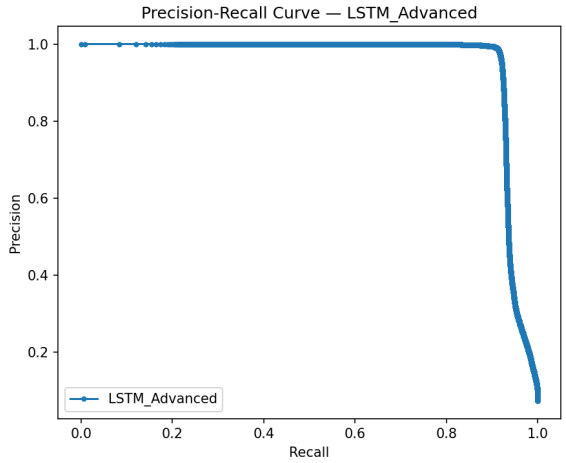
6.2.2 Precision-Recall krivky



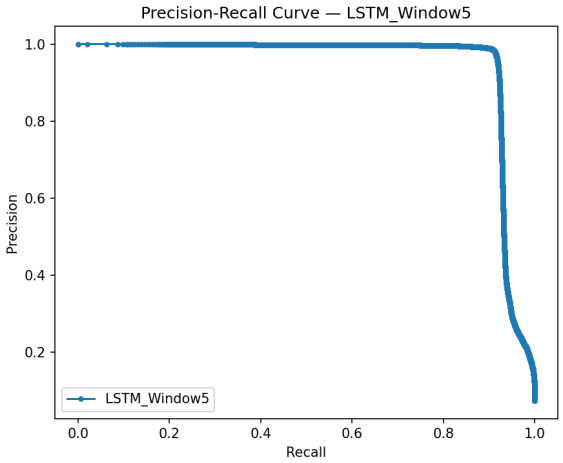
Obr. 5: PR krivka — E1: MLP Baseline, AP = 0.9941



Obr. 6: PR krivka — E2: MLP Large, AP = 0.9942



Obr. 7: PR krivka — E3: LSTM Win=10, AP = 0.9482



Obr. 8: PR krivka — E4: LSTM Win=5, AP = 0.9450

MLP krivky zostávajú blízko Precision ≈ 1.0 až do Recall ≈ 0.8. LSTM krivky majú odlišný tvar — LSTM Win=5 má Precision $= 0.99$ pri $\theta = 0.5$ oproti MLP Precision $= 0.91$, čo vysvetľuje nižší FPR LSTM.

6.3 Výsledky na NSL-KDD

Testovacia množina obsahuje 22 543 vzoriek (9 710 Normal, 12 833 Attack). Per-class ordered split: tréning 100 778 vzoriek (80 %), validácia 25 195 vzoriek (20 %), testovacia množina je originálny `KDDTest+.csv` s unseen attack typmi.

Model	ROC-AUC	Avg. Prec.	F1	FNR	FP	FN
E5: MLP Baseline (64-32)	0.9463	0.9502	0.7628	36.83 %	313	4 727
E5b: MLP Large (128-64, Drop=0.3)	**0.9503**	**0.9603**	**0.7983**	**32.27 %**	**251**	**4 141**
E6: LSTM Baseline (lstm=64)	0.8651	0.9042	0.7604	34.72 %	824	4 454
E6b: LSTM Large (lstm=128, D=32)	0.8857	0.9172	0.7350	39.81 %	460	5 107

Tabuľka 13: Výsledky na testovacej množine NSL-KDD (22 543 vzoriek). Tučné = najlepší výsledok.

E5 vs. E5b — vplyv kapacity MLP. Na rozdiel od CIC-IDS2017 (saturácia, H2), na NSL-KDD MLP Large (E5b) výrazne prekonala Baseline (E5) — potvrdzuje hypotézu H1 (underfitting):

$$\Delta\text{F1} = +0.0355, \qquad \Delta\text{FNR} = -4.56\,\text{pp}, \qquad \Delta\text{FN} = -586\,(-12.4\,\%) \tag{27}$$

41-dimenzionálny vstupný priestor so štyrmi typmi útokov (DoS, Probe, R2L, U2R) s výrazne odlišnými profilmi vyžaduje väčšiu latentú kapacitu, než poskytuje architektúra $(64, 32)$. Toto správanie je presne opačné oproti CIC-IDS2017, kde MLP Large neprospel.

E6 vs. E6b — vplyv kapacity LSTM. LSTM Large (E6b) dosiahlo *horšie* výsledky ako LSTM Baseline (E6):

$$\Delta\text{F1} = -0.0254, \qquad \Delta\text{FNR} = +5.09\,\text{pp}, \qquad \Delta\text{FN} = +653\,(+14.7\,\%) \tag{28}$$

Väčšia kapacita LSTM aktívne škodí: model "memoruje" pseudotemporálne vzory v náhodne zoradených záznamoch, čo vedie k horšej generalizácii na unseen attack typy. Väčší LSTM je teda kontraproduktívny na i.i.d. datasete.

Off-by-one bug — LSTM NSL-KDD. Pôvodná implementácia obsahovala kritickú chybu priradenia mítky:

$$\text{(chybné)} \quad y[i+T] \quad \longrightarrow \quad \text{(opravené)} \quad y[i+T-1] \tag{29}$$

Táto chyba viedla k Accuracy = 0.48, F1 = 0.49 — hodnoty blízke náhodnému hádaniu. Po oprave LSTM dosiahol Accuracy = 0.77, porovnateľné s MLP.

6.4 Súhrnné porovnanie všetkých experimentov

Exp.	Model	Dataset	Acc.	F1	ROC-AUC	FNR	Avg. Prec.
E1	MLP Baseline (64-32)	CIC	0.98	0.9504	0.9985	0.55 %	0.9941
E2	MLP Large (128-64)	CIC	0.98	0.9490	0.9986	0.60 %	0.9942
E3	LSTM Win=10	CIC	0.99	0.9477	0.9821	9.24 %	0.9482
E4	LSTM Win=5	CIC	0.99	0.9455	0.9825	9.10 %	0.9450
E5	MLP Baseline (64-32)	NSL	0.78	0.7628	0.9463	36.83 %	0.9502
E5b	MLP Large (128-64)	NSL	—	**0.7983**	**0.9503**	**32.27 %**	0.9603
E6	LSTM Baseline (lstm=64)	NSL	0.77	0.7604	0.8651	34.72 %	0.9042
E6b	LSTM Large (lstm=128)	NSL	0.75	0.7350	0.8857	39.81 %	0.9172

Tabuľka 14: Súhrnné porovnanie všetkých ôsmich experimentov (CIC: per-day split; NSL: per-class ordered split)

7 Kritická analýza a diskusia

7.1 Porovnávanie architektúr

MLP — silné stránky a limitácie. MLP spracúva každý sieťový tok ako izolovanú vzorku $\mathbf{x} \in \mathbb{R}^{69}$, čím úplne ignoruje časový kontext. Napriek tomu dosiahol ROC-AUC = 0.9985 a FNR = 0.55 % na CIC-IDS2017. Dôvodom je charakter datasetu: väčšina zastúpených typov útokov sa prejavuje extrémne odlišnými hodnotami štatistických príznakov aj v jednotlivých tokoch (DoS/DDoS: extrémne `Flow Bytes/s`; Brute Force: vysoký `Fwd Packets/s`; Port Scan: nízke `Flow Duration`). Limitáciou MLP zostáva neschopnosť zachytiť viacfázové útoky rozložené cez desiatky tokov (Infiltration, pomalý Botnet).

LSTM — vplyv per-day stratifikovaného splitu. CIC-IDS2017 je *scenárový dataset* — každý deň obsahuje odlišné typy útokov:

Deň	Typy útokov	Globálny split	Per-day split
Pondelok	iba BENIGN	Train	Train + Val + Test
Utorok	FTP-Patator, SSH-Patator	Train	Train + Val + Test
Streda	DoS Hulk, GoldenEye, Slowloris	Train	Train + Val + Test
Štvrtok	WebAttacks, Infiltration	Val	Train + Val + Test
Piatok	Botnet, PortScan, DDoS	Test	Train + Val + Test

Tabuľka 15: Rozdelenie typov útokov pri globálnom vs. per-day splite

Deň / súbor	Train	Val	Test
Monday (iba BENIGN)	338 867	84 717	105 897
Tuesday (FTP-Patator, SSH)	285 212	71 304	89 129
Wednesday (DoS, GoldenEye)	442 499	110 625	138 282
Thursday-Morning (WebAttacks)	108 947	27 237	34 047
Thursday-Afternoon (Infiltration)	184 572	46 144	57 679
Friday-Morning (Botnet)	122 183	30 545	38 183
Friday-Afternoon-DDoS	144 455	36 113	45 143
Friday-Afternoon-PortScan	183 101	45 775	57 220
Spolu	**1 809 836**	**452 460**	**565 580**

Tabuľka 16: Per-day stratifikovaný split — počty riadkov z každého dňa

Epocha	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Val Accuracy	Trend
1	0.168	0.093	0.961	konvergencia
2	0.084	0.067	0.974	konvergencia
5	0.050	0.281	0.921	val_loss začína rásť
7	0.005	0.931	0.789	divergencia
12	0.004	1.663	0.786	silný overfit
15	0.003	1.635	0.811	bez konvergencie

Tabuľka 17: Priebeh tréningu LSTM Advanced — divergencia strát pri globálnom splite

Dôkaz divergencie pri globálnom splite. Tréningová strata klesla o 98 % (0.168 → 0.003), zatiaľ čo validačná strata vzrástla o 1 658 % (0.093 → 1.635). Model sa naučil rozoznávať DoS

a Brute Force sekvencie s takmer dokonalou presnosťou, ale stratil schopnosť generalizovať na WebAttacks a Infiltration.

7.2 Analýza výsledkov ablačnej štúdie

Model	ROC-AUC	Avg. Prec.	F1	Záver
CIC-IDS2017				
E1: MLP Baseline	0.998549	0.994096	0.950431	referencia
E2: MLP Large	0.998582	0.994218	0.949043	F1 mierne horšie — H2 potvrdená
NSL-KDD				
E5: MLP Baseline	0.9463	0.9502	0.7628	referencia
E5b: MLP Large	0.9503	0.9603	0.7983	F1 výrazne lepšie — H1 potvrdená

Tabuľka 18: MLP ablačná štúdia kapacity — porovnanie pre oba datasety

E1 vs. E2 (CIC) a E5 vs. E5b (NSL) — vplyv kapacity MLP. Na CIC-IDS2017 architektúra $(64, 32)$ dosiahla saturačný bod (ΔF1 $= -0.001$, ΔFN $= +62$). Na NSL-KDD väčšia sieť zmešká o 586 útokov menej ($-12.4\,\%$, ΔF1 $= +0.036$) — presne opačné správanie. Vysvetlením je rozdielna komplexita príznakového priestoru: 69-dimenzionálny CIC-IDS2017 s monotonickými útočnými vzormi saturuje malý model rýchlejšie ako 41-dimenzionálny NSL-KDD s heterogénnymi štyrmi typmi útokov.

Model	ROC-AUC	Avg. Prec.	F1	FNR
E3: LSTM Win=10	0.9821	0.9482	0.9477	9.24 %
E4: LSTM Win=5	0.9825	0.9450	0.9455	9.10 %
Δ	+0.0004	−0.0032	−0.0022	≈ 0

Tabuľka 19: Porovnanie LSTM Window=10 vs. Window=5 (per-day split)

E3 vs. E4 — vplyv dĺžky časového okna LSTM. ΔF1 $= 0.0022$ zodpovedá $\sim$12 vzorkám z 565 576 testovacích. Pre produkčné nasadenie **odporúčame Window=5**: o 50 % menšia alokácia RAM ($\approx$ 2.5 GB úspora) a nižšia latencia inferencie pri identickom výkone.

E6 vs. E6b — vplyv kapacity LSTM na NSL-KDD. Väčší LSTM (E6b) aktívne škodí na i.i.d. datasete — zmešká o 653 útokov viac (+14.7 %). NSL-KDD záznamy sú bez garantovaného chronologického poriadku, preto LSTM "memoruje" pseudotemporálne vzory v náhodne zoradených záznamoch, čo vedie k horšej generalizácii na unseen attack typy.

Experiment	Dataset	Δ F1	Δ FNR	Záver
E1 $\rightarrow$ E2 (MLP)	CIC	−0.001	+0.05 pp	saturácia
E5 $\rightarrow$ E5b (MLP)	NSL	+0.036	−4.56 pp	underfitting
E3 $\rightarrow$ E4 (LSTM)	CIC	−0.002	−0.14 pp	ekvivalentné
E6 $\rightarrow$ E6b (LSTM)	NSL	−0.025	+5.09 pp	overfit

Tabuľka 20: Porovnanie vplyvu kapacity modelu naprieč datasetmi

Súhrnný prehľad — vplyv kapacity naprieč datasetmi.

7.3 Analýza chýb

Model	FN	FNR	FP	FPR
E1: MLP Baseline	611	0.55 %	10 936	2.41 %
E2: MLP Large	673	0.60 %	11 208	2.47 %
E3: LSTM Win=10	3 893	9.24 %	330	0.06 %
E4: LSTM Win=5	3 838	9.10 %	581	0.11 %

Tabuľka 21: Analýza chýb na testovacej množine CIC-IDS2017 — per-day split

Kvantitatívna analýza FP a FN — CIC-IDS2017.

Model	FN	FNR	FP	FPR
E5: MLP Baseline	4 727	36.83 %	313	3.22 %
E5b: MLP Large	4 141	32.27 %	251	2.58 %
E6: LSTM Baseline	4 454	34.72 %	824	8.49 %
E6b: LSTM Large	5 107	39.81 %	460	4.74 %

Tabuľka 22: Analýza chýb na testovacej množine NSL-KDD

Kvantitatívna analýza FP a FN — NSL-KDD. Kľúčovým pozorovaním na NSL-KDD je asymetria FNR vs. FPR pre všetky modely: FPR $\approx$ 2.5–8.5 % (legitímna komunikácia identifikovaná spoľahlivo), avšak FNR $\approx$ 32–40 % (každý tretí útok zmeškaný). Táto asymetria nie je spôsobená nedostatočnou kapacitou modelov, ale štruktúrou NSL-KDD: 32–40 % útokov patrí k typom (R2L, U2R varianty) neprítomným v tréningovej množine. Porovnanie s CIC-IDS2017 odhaľuje $\approx 59\times$ zhoršenie FNR (0.55 % vs. 32.3 % pri najlepšom modeli E5b) — ilustruje zásadné obmedzenie supervízovaného učenia pri out-of-distribution útokoch.

7.4 Technické poznatky z HPC nasadenia

Pamäťová náročnosť LSTM sekvencií. Celková spotreba RAM pri tréningu LSTM na CIC-IDS2017 závisí od veľkosti časového okna a spôsobu alokácie:

Komponent	Window=10	Window=5
Sekvencie X (train + val + test), float32	6.80 GB	3.41 GB
Raw arrays po MinMaxScaler (float32)	0.75 GB	0.75 GB
OS + Python + Pandas (načítanie CSV)	$\approx$ 3.50 GB	3.50 GB
Spike pri `np.array(list)` (dočasný)	+5.00 GB	+2.50 GB
Peak RAM celkovo	$\approx$ 16–18 GB	$\approx$ 9–11 GB

Tabuľka 23: Pamäťová náročnosť LSTM tréningu na CIC-IDS2017

Dočasný spike vzniká pri volaní `np.array(list_of_arrays)` v `create_sequences`, kde Python list a výsledný NumPy array existujú v pamäti zároveň. Eliminuje sa priamou prealokáciou:

```
X_win = np.empty((N, window_size, F), dtype=np.float32)
y_win = np.empty(N, dtype=np.float32)
for i in range(N):
```

```
X_win[i] = X[i:i + window_size]
y_win[i] = y[i + window_size - 1]
```

Na lokálnom stroji s 16 GB RAM tréning Window=10 spôsobil `Killed` signal od OS (OOM). H200 s 143 GB VRAM túto záťaž zvládol bez obmedzení, čo potvrdzuje nevyhnutnosť HPC infraštruktúry pre Window=10. Na Google Colab Free (12.7 GB): Window=10 spôsobí OOM, Window=5 s prealokáciou ($\approx$ 9–11 GB peak) je tesne realizovateľný.

Etapa vývoja	**ROC-AUC**	**F1**	**FNR**
Off-by-one bug + shuffle=False	≈ 0.500	≈ 0	$\approx 100\,\%$
Opravený bug + globálny chrono. split	0.9029	0.681	$\sim 21\,\%$
Opravený bug + per-day split (finálny)	**0.9821**	**0.9477**	$\sim$ **9** %
MLP Baseline (shuffle=True, bez bugu)	0.9985	0.9504	0.55 %

Tabuľka 24: Evolúcia výsledkov LSTM Advanced počas vývoja experimentov

Evolúcia výsledkov LSTM počas vývoja.

8 Obmedzenia a možnosti ďalšej práce

8.1 Obmedzenia

- **Offline dataset:** CIC-IDS2017 je statický — modely nie sú testované na schopnosť adaptácie na nové typy útokov (concept drift).
- **Binárna klasifikácia:** projekt nerieši viactriednu klasifikáciu (identifikáciu konkrétneho typu útoku).
- **Sekvenčné okná LSTM:** prístup predpokladá, že záznamy v CSV sú chronologicky zoradené vnútri každého súboru, čo nie je explicitne garantované (stĺpec `Timestamp` nebol použitý pri zoraďovaní).
- **Pamäťová náročnosť:** Window=10 vyžaduje 16–18 GB peak RAM — nevykonateľné na Colab Free (12.7 GB). Window=5 s priamou prealokáciou je realizovateľný na $\approx$10 GB.
- **Absencia real-time evaluácie:** výkon v reálnom prostredí s nízkou latenciou nebol meraný.

8.2 Možnosti ďalšej práce

- **Viactriedna klasifikácia:** rozšírenie na identifikáciu konkrétneho typu útoku (DoS, DDoS, Brute Force...).
- **Transformer architektúra:** nahradenie LSTM modernými self-attention mechanizmami, ktoré lepšie spracúvajú dlhé sekvencie.
- **Autoencoder pre anomáliu detekciu:** namiesto supervízneho učenia trénovať len na legitímnom provoze a detekovať odchýlky.
- **Online learning:** adaptácia modelu na nové typy útokov bez úplného pretrénovania.
- **SHAP analýza:** identifikácia najdôležitejších príznakov pre interpretovateľnosť rozhodnutí modelu.

- **Optimalizácia klasifikačného prahu:** systematická analýza prahu θ minimalizujúca $C_{\text{FN}} \cdot \text{FN} + C_{\text{FP}} \cdot \text{FP}$.

9 Príspevok jednotlivých členov tímu

Člen tímu	Zodpovednosť	Konkrétne výstupy
Andrii Sysak	Predspracovanie dát, pipeline načítania	`data_preprocesing.py`, per-day split, per-class split
Oleksandr Zemlyanskiy	Návrh a implementácia MLP a LSTM architektúr	`models.py`, `lstm.py`, oprava off-by-one bugu
Nikita Synenko	Tréning, evaluácia, vizualizácia výsledkov	`train_eval.py`, confusion matrix grafy, PR krivky
Andrii Rytskyi	Metodológia, ablation study, HPC nasadenie, správa	`main.py`, SLURM skripty, report, README

Tabuľka 25: Príspevok členov tímu

10 Zoznam použitej literatúry

Literatúra

[1] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In ICISSP (pp. 108-116).

[2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.

[4] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. ICML.

[5] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. JMLR, 15(1), 1929-1958.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825-2830.

[7] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. OSDI, 16, 265-283.